



To bee or not to bee

Classification d'insectes pollinisateurs par apprentissage automatique

Rapport de projet — Machine Learning

Cours : IG.2412

Juin 2026

Auteur : TCHOUMBOU FONGANG Khaleb Darius

Table des matières

1. Introduction.....	3
2. Description des données.....	3
2.1. Distribution des classes.....	3
2.2. Aperçu visuel du dataset.....	4
2.3. Difficulté intrinsèque du problème.....	6
3. Extraction de caractéristiques (Features).....	6
3.1. Caractéristiques obligatoires.....	6
3.1.1. Ratio de pixels (bug / image).....	6
3.1.2. Statistiques RGB dans le masque.....	6
3.1.3. Symétrie de forme.....	6
3.1.4. Symétrie de couleur.....	6
3.2. Caractéristiques additionnelles (choix libre).....	7
3.2.1. Statistiques HSV (couleur perceptuelle).....	7
3.2.2. Texture GLCM (Gray-Level Co-occurrence Matrix).....	7
3.2.3. Descripteurs de forme.....	7
3.2.4. Couleur du fond.....	7
3.3. Récapitulatif du vecteur de features.....	7
4. Visualisation et exploration.....	8
4.1. Distribution des classes.....	8
4.2. Projection linéaire — PCA.....	8
4.3. Projections non linéaires.....	9
5. Algorithmes d'apprentissage testés.....	11
5.1. Méthodologie d'évaluation : validation croisée stratifiée.....	11
5.2. Méthodes supervisées non-ensemble.....	11
5.2.1. Support Vector Machine (SVM) avec noyau RBF.....	11
5.2.2. K-Nearest Neighbors (KNN, k=5).....	11
5.3. Méthode d'ensemble.....	11
5.3.1. Random Forest.....	11
5.4. Méthodes de clustering.....	11
5.4.1. K-Means.....	12

5.4.2. Agglomerative Clustering (hiérarchique)	12
5.4.3. Métriques d'évaluation du clustering	12
5.5. Deep Learning (point bonus) — ResNet18 par transfert	12
5.5.1. Architecture.....	12
5.5.2. Hyperparamètres choisis avec soin.....	12
6. Optimisation des hyperparamètres (Tuning)	14
6.1. Grilles testées	14
6.2. Métrique d'optimisation	14
6.3. Meilleurs hyperparamètres retenus	14
7. Étude d'ablation : les features de fond	15
7.1. Résultats comparés	15
7.2. Interprétation	15
8. Résultats globaux	16
8.1. Tableau comparatif des modèles	16
8.2. Analyse par classe du modèle final	16
8.3. Résultats du clustering	17
8.4. Importance des features (modèle Random Forest)	18
9. Modèle final retenu.....	20
9.1. Pipeline final.....	20
9.2. Justification du choix.....	20
10. Limites et pistes d'amélioration	20
11. Conclusion	21

1. Introduction

Ce rapport présente la méthodologie et les résultats d'un projet de classification supervisée d'insectes pollinisateurs à partir d'images haute résolution accompagnées de leurs masques de segmentation. L'objectif est de construire un modèle capable de distinguer plusieurs catégories d'insectes (abeilles, bourdons, papillons, syrphes, guêpes, libellules) en s'appuyant sur des caractéristiques visuelles extraites de l'insecte isolé du fond grâce au masque.

Le dataset fourni se compose de 347 images au total, dont 250 destinées à l'entraînement (avec labels) et 97 au test (sans labels, prédictions à fournir). Chaque image est accompagnée d'un masque binaire qui isole précisément l'insecte du fond, ce qui permet de calculer des caractéristiques directement sur la zone d'intérêt. Cette mise à disposition d'un masque est un atout important : elle permet d'éviter que le modèle apprenne à reconnaître le décor plutôt que l'insecte (effet « Clever Hans »).

Le projet a été structuré autour de quatre étapes principales : (1) extraction de caractéristiques numériques décrivant la couleur, la forme et la texture de chaque insecte, (2) visualisation et exploration du dataset via des projections dimensionnelles, (3) test et comparaison de plusieurs algorithmes d'apprentissage (supervisés non-ensemble, ensemble, clustering, et un modèle de Deep Learning par transfert), (4) sélection du meilleur modèle pour produire les prédictions finales sur le test set.

2. Description des données

Après chargement du fichier de labels et inspection du dataset, une image (identifiant 154) a dû être écartée car son masque de segmentation était absent. Le dataset effectif d'entraînement compte donc 249 images réparties en six classes.

2.1. Distribution des classes

Le tableau ci-dessous présente la répartition des 249 images d'entraînement :

Classe	Effectif	Proportion	Catégorie
Bee (abeille)	115	46,2 %	Majoritaire
Bumblebee (bourdon)	100	40,2 %	Majoritaire
Butterfly (papillon)	15	6,0 %	Minoritaire
Hover fly (syrphe)	9	3,6 %	Minoritaire
Wasp (guêpe)	9	3,6 %	Minoritaire
Dragonfly (libellule)	1	0,4 %	Rare

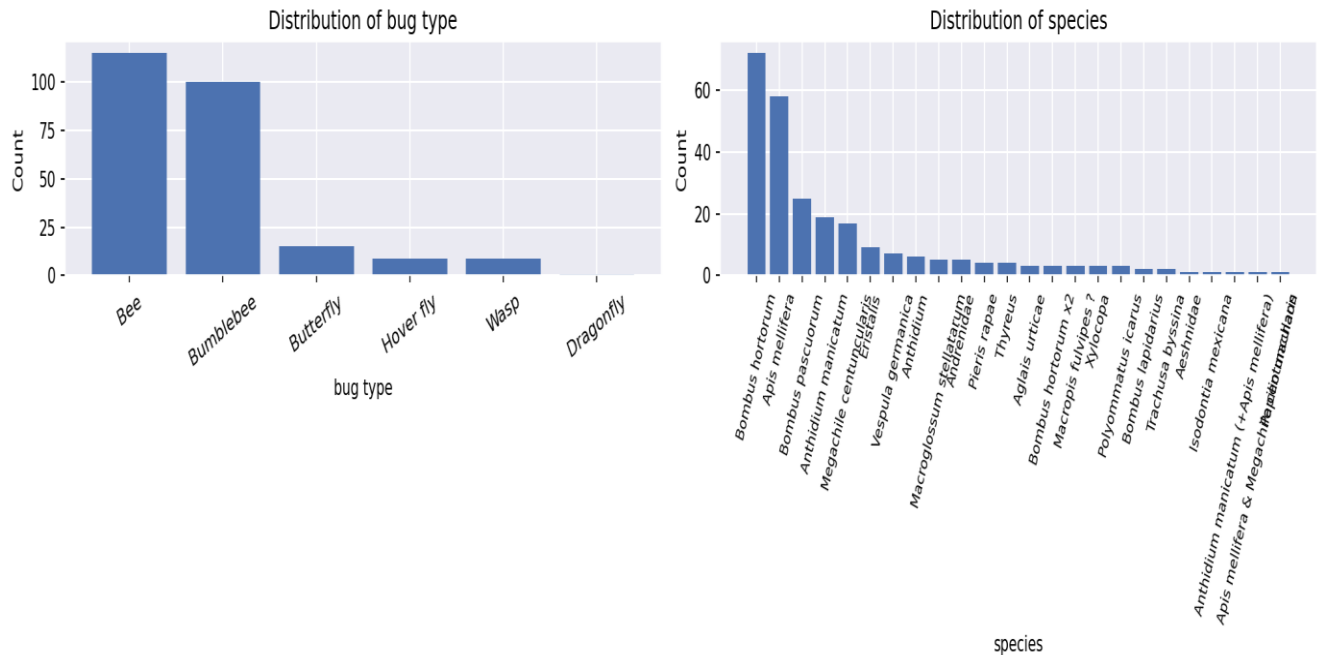


Figure 1. Distribution des classes (gauche) et des espèces (droite) sur les 249 images d'entraînement.

Cette distribution révèle deux problèmes structurels du dataset. D'une part, un déséquilibre marqué : les classes Bee et Bumblebee représentent à elles seules plus de 86 % des images, tandis que Hover fly, Wasp et Dragonfly réunissent à peine 8 %. D'autre part, la classe Dragonfly n'est représentée que par une seule image, ce qui rend impossible l'apprentissage statistique fiable de cette catégorie et la rend inéligible à une validation croisée stratifiée.

Pour gérer ce déséquilibre, deux jeux de données ont été préparés en parallèle : `df_eval` (classes avec au moins 5 échantillons, soit 248 images) pour les évaluations en validation croisée, et `df_full` (totalité des 249 images) pour l'entraînement du modèle final déployé sur le test set.

2.2. Aperçu visuel du dataset

La figure ci-dessous présente quatre exemples d'abeilles avec leurs masques de segmentation associés. On observe que les abeilles sont souvent photographiées sur des fleurs blanches (digitales) et occupent une faible portion de l'image. Le masque binaire isole précisément le contour de l'insecte, ce qui permet d'extraire les features uniquement sur la zone d'intérêt.

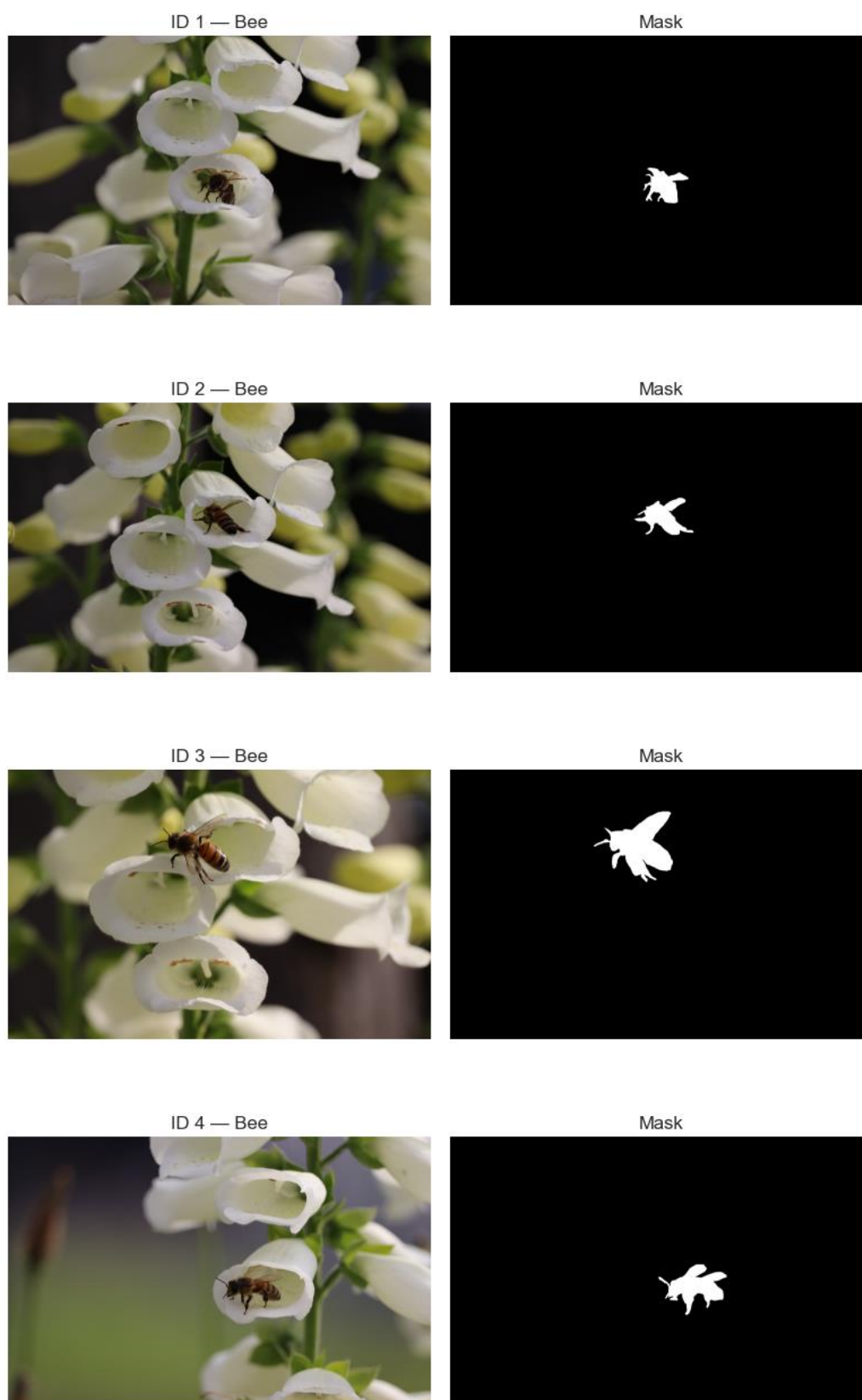


Figure 2. Quatre exemples d'images d'entraînement (abeilles, ID 1 à 4) avec leur masque de segmentation associé.

2.3. Difficulté intrinsèque du problème

Au-delà du déséquilibre, le problème présente une difficulté biologique réelle : les syrphes (Hover fly) pratiquent ce que l'on appelle le mimétisme batésien — ils ont évolué pour ressembler aux abeilles afin de tromper leurs prédateurs. Cette ressemblance visuelle se traduit dans les features : un syrphe partage souvent les couleurs jaunes et noires des abeilles, des dimensions comparables, et une silhouette similaire. Cette ambiguïté visuelle constitue une limite fondamentale qu'aucun modèle ne pourra entièrement contourner sans recourir à des informations plus fines (texture du corps, structure des ailes, antennes).

3. Extraction de caractéristiques (Features)

Le cahier des charges du projet impose l'extraction d'au moins sept caractéristiques, dont certaines sont explicitement listées. Nous sommes allés au-delà du minimum requis en construisant un vecteur de descripteurs riche couvrant trois familles complémentaires : la couleur, la forme et la texture.

3.1. Caractéristiques obligatoires

3.1.1. Ratio de pixels (bug / image)

Le ratio de pixels correspond à la proportion de pixels appartenant à l'insecte par rapport à la surface totale de l'image. Il s'agit d'un proxy de la taille relative de l'insecte dans la photo. Cette caractéristique permet par exemple de séparer un papillon (souvent grand, déployé sur une fleur) d'une abeille photographiée de loin.

3.1.2. Statistiques RGB dans le masque

Pour chacun des trois canaux Rouge, Vert et Bleu, nous calculons cinq statistiques sur les seuls pixels appartenant à l'insecte (donc à l'intérieur du masque) : minimum, maximum, moyenne, médiane et écart-type. Cela représente 15 features (3 canaux $\times$ 5 statistiques). Ces descripteurs capturent à la fois la teinte dominante de l'insecte (moyenne, médiane) et la diversité chromatique (écart-type, écart min-max).

3.1.3. Symétrie de forme

La symétrie de forme évalue dans quelle mesure le masque de l'insecte est symétrique par rapport à son axe vertical médian. Concrètement, nous comparons le masque à son miroir et calculons la proportion de pixels qui coïncident. Un papillon aux ailes déployées affichera un score élevé, tandis qu'une abeille vue de profil aura un score plus faible.

3.1.4. Symétrie de couleur

Sur le même principe que la symétrie de forme, mais appliquée à l'image couleur : on compare les pixels gauche et droit (par rapport à l'axe vertical) et on mesure la similarité moyenne de leurs valeurs RGB. Cette métrique complète la symétrie de forme en capturant les motifs colorés (ailes de papillon, bandes jaunes-noires).

3.2. Caractéristiques additionnelles (choix libre)

Au-delà des features imposées, nous avons enrichi le vecteur de description avec quatre familles supplémentaires destinées à mieux capturer les différences subtiles entre classes.

3.2.1. Statistiques HSV (couleur perceptuelle)

L'espace HSV (Hue, Saturation, Value) sépare la teinte de la luminosité, ce que RGB ne fait pas. Une abeille jaune-noire et un syrphe jaune-noir auront des moyennes de teinte (H) très proches, mais la saturation (S) et la valeur (V) peuvent révéler des nuances. Nous calculons la moyenne et l'écart-type des trois canaux HSV à l'intérieur du masque (6 features).

3.2.2. Texture GLCM (Gray-Level Co-occurrence Matrix)

La GLCM décrit la texture d'une image en comptant la fréquence des paires de pixels voisins ayant certaines combinaisons de niveaux de gris. Nous en extrayons quatre statistiques classiques : contraste (variations locales), dissimilarité (différences entre pixels voisins), homogénéité (lissé de la texture), et corrélation (cohérence spatiale). Ces features distinguent par exemple les ailes lisses d'une abeille des motifs structurés d'un papillon.

3.2.3. Descripteurs de forme

Cinq descripteurs géométriques sont calculés à partir du masque : la compacité ($\text{perimeter}^2 / \text{aire}$ — élevée pour les formes allongées), la solidity ($\text{aire} / \text{aire de l'enveloppe convexe}$ — basse pour les formes très découpées), l'eccentricité (allongement de l'ellipse englobante), l'extent ($\text{aire} / \text{aire de la bounding box}$) et l'aspect ratio (largeur / hauteur). Ces descripteurs complètent la symétrie en quantifiant l'allure générale de l'insecte.

3.2.4. Couleur du fond

Bien que les masques nous permettent d'isoler l'insecte, nous avons également extrait la couleur moyenne du fond (RGB en dehors du masque). Cette feature encode le contexte écologique : un insecte photographié sur une fleur jaune vs sur un fond vert d'herbe. Nous reviendrons sur l'utilité — controversée — de ces features dans la section 7 (étude d'ablation).

3.3. Récapitulatif du vecteur de features

Famille	Nombre	Statut
Ratio de pixels	1	Obligatoire
Statistiques RGB dans le masque	15	Obligatoire
Symétrie de forme et de couleur	2	Obligatoire
Statistiques HSV	6	Choix libre
Texture GLCM	4	Choix libre
Descripteurs de forme	5	Choix libre
Couleur moyenne du fond	3	Choix libre
Total	38	—

4. Visualisation et exploration

Avant de passer à la modélisation, il est crucial d'explorer visuellement le dataset en projetant le vecteur de features dans un espace de dimension réduite (2D). Cette étape permet d'estimer la séparabilité intrinsèque des classes et d'identifier d'éventuels problèmes de structure des données.

4.1. Distribution des classes

Un premier graphique en barres montre la distribution déséquilibrée déjà discutée. Cette visualisation est essentielle car elle conditionne toutes les décisions ultérieures : choix d'utiliser des poids de classe inversement proportionnels à la fréquence, choix de la métrique d'évaluation (F1 macro donnerait un poids égal à chaque classe ; F1 weighted privilégie les classes majoritaires).

4.2. Projection linéaire — PCA

L'Analyse en Composantes Principales (PCA) projette les données dans un sous-espace linéaire qui maximise la variance préservée. Sur les deux premières composantes, les classes Bee et Bumblebee occupent des régions partiellement séparées mais avec un chevauchement important. Les Hover fly se confondent en grande partie avec les Bee, ce qui confirme visuellement l'hypothèse du mimétisme batésien. Quelques Butterfly se détachent nettement à droite, suggérant qu'au moins une partie de cette classe est intrinsèquement séparable.

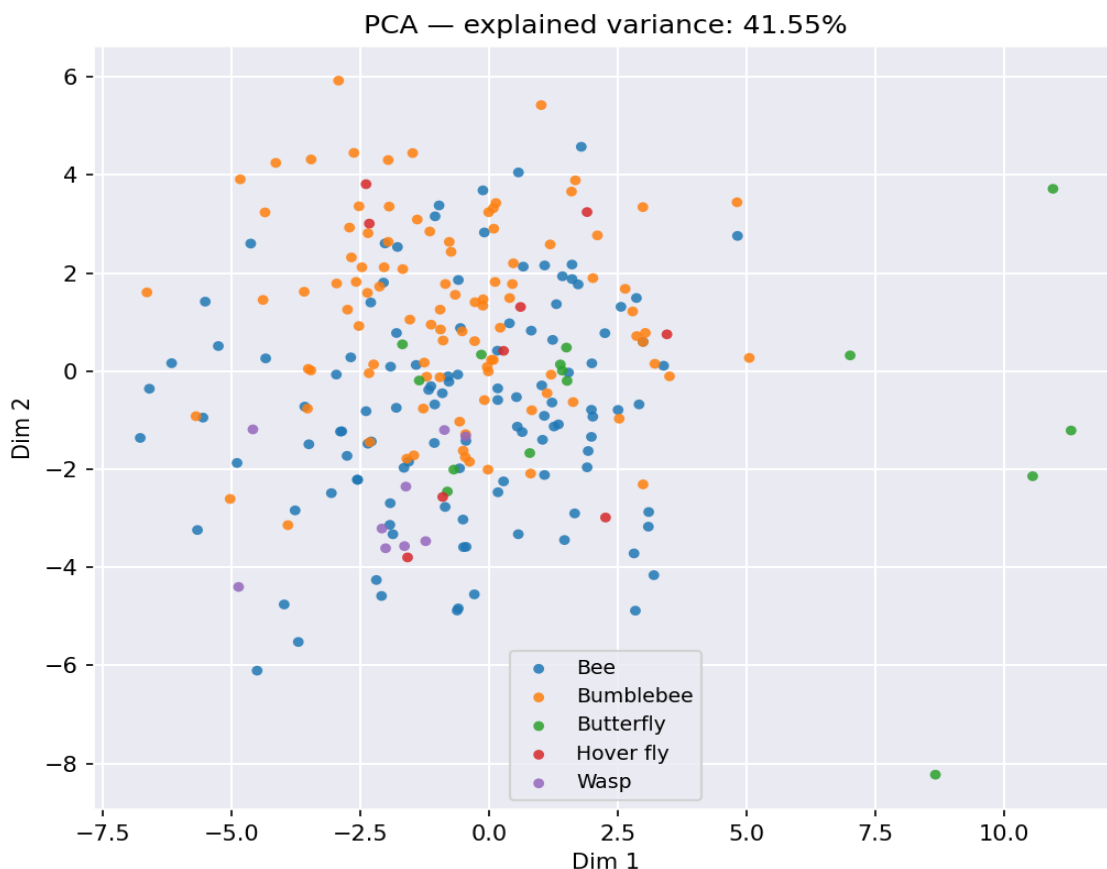


Figure 3. Projection PCA sur les deux premières composantes (41,55 % de variance expliquée).

4.3. Projections non linéaires

Trois méthodes non linéaires complémentaires ont été utilisées pour révéler des structures que la PCA pourrait masquer :

- **t-SNE** : préserve les voisinages locaux. Produit généralement des amas compacts par classe quand les données sont séparables. Sur notre dataset, on observe des amas Bee et Bumblebee bien formés, mais les classes minoritaires se dispersent.
- **UMAP** : alternative récente à t-SNE, plus rapide et préservant mieux la structure globale. Confirme les conclusions de t-SNE.

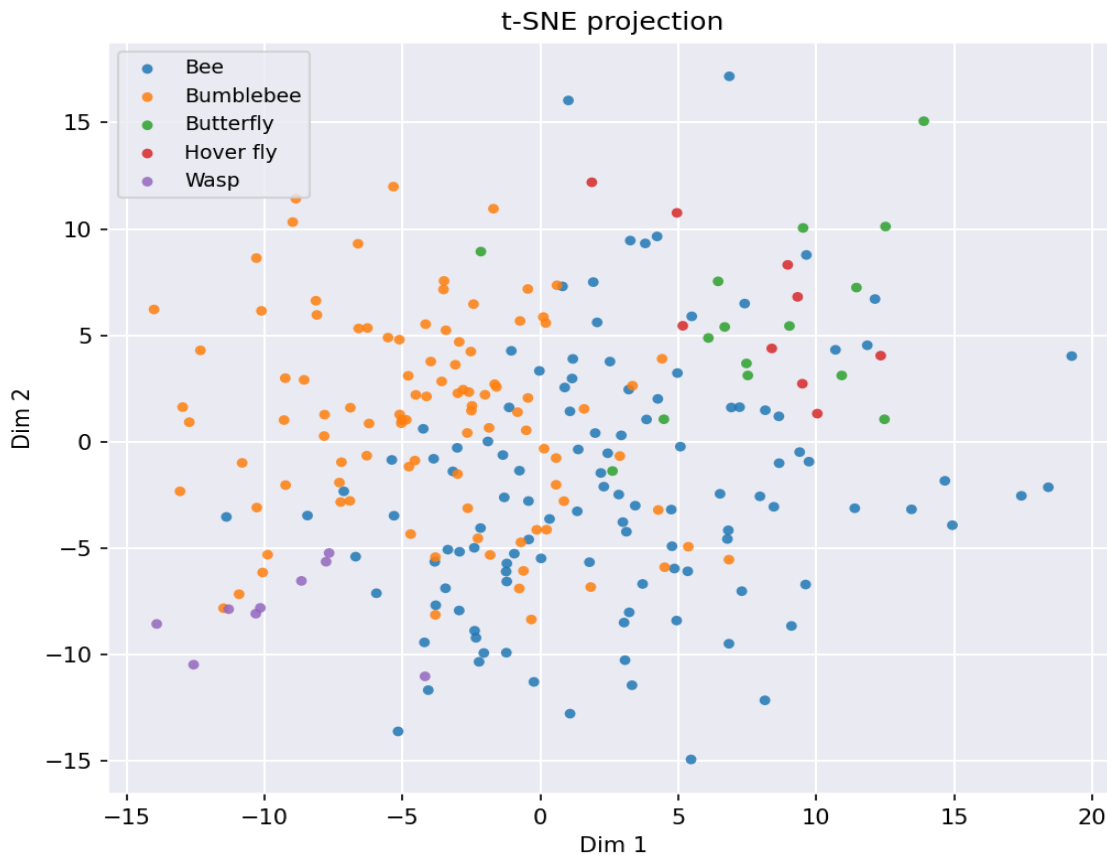


Figure 4. Projection t-SNE — les classes Bee et Bumblebee forment des zones partiellement distinctes ; les minoritaires sont éparpillées.

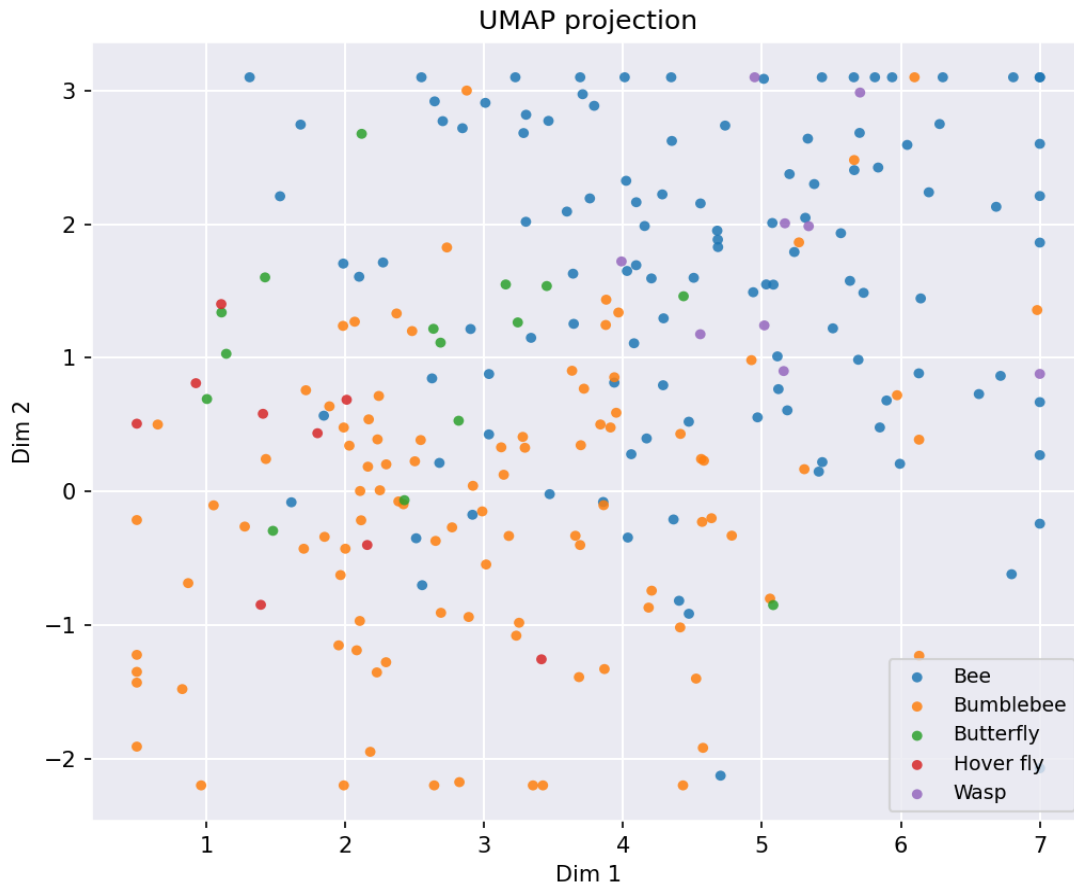


Figure 5. Projection UMAP — confirme la structure observée en t-SNE, avec une séparation modérée entre Bee et Bumblebee.

Pour les deux méthodes non linéaires, les classes majoritaires apparaissent comme deux régions distinctes (avec un léger chevauchement), tandis que les classes minoritaires sont dispersées au sein ou entre ces régions. Cette observation visuelle anticipe les difficultés rencontrées par la suite : un modèle pourra bien classer Bee et Bumblebee mais aura nécessairement plus de mal avec Hover fly et Wasp.

Un point méthodologique important : avant chaque projection, les features ont été standardisées (moyenne nulle, écart-type unitaire) via StandardScaler. Cette étape est indispensable car les features ont des échelles très différentes (un ratio de pixels entre 0 et 1, des valeurs RGB entre 0 et 255, des compacités potentiellement supérieures à 100).

5. Algorithmes d'apprentissage testés

Conformément au cahier des charges, nous avons testé au moins deux méthodes supervisées non-ensemble, une méthode d'ensemble, deux méthodes de clustering, et — pour le point bonus — une méthode de Deep Learning par transfert d'apprentissage.

5.1. Méthodologie d'évaluation : validation croisée stratifiée

Toutes les évaluations ont été réalisées par validation croisée stratifiée 5-fold avec mélange des indices (shuffle=True, random_state=0). Le choix de la stratification est crucial dans notre cas pour garantir que chaque fold contienne une proportion similaire de chaque classe. L'absence de stratification — ou même l'absence de mélange — donnerait des résultats extrêmement instables car le dataset est ordonné par espèce.

Un incident pédagogiquement intéressant a confirmé ce point : un premier essai de GridSearchCV avec cv=5 (StratifiedKFold non mélangé par défaut) produisait des scores artificiellement bas (Random Forest passant de 0,82 à 0,70 d'accuracy). Le passage à un StratifiedKFold(shuffle=True) explicite a restauré des résultats cohérents et reproductibles.

5.2. Méthodes supervisées non-ensemble

5.2.1. Support Vector Machine (SVM) avec noyau RBF

Le SVM cherche un hyperplan qui maximise la marge entre les classes. Avec un noyau RBF (Radial Basis Function), il peut tracer des frontières non linéaires arbitrairement complexes. C'est l'algorithme qui s'est révélé le plus performant sur notre problème. Configuration de base : C=1, gamma='scale', class_weight='balanced' pour compenser le déséquilibre.

5.2.2. K-Nearest Neighbors (KNN, k=5)

Le KNN classe un point par vote majoritaire de ses k plus proches voisins dans l'espace des features standardisées. Algorithme simple, sans phase d'apprentissage, qui sert de référence rapide. Configuration : k=5 voisins, distance euclidienne. Performant si les classes forment des amas compacts dans l'espace des features, ce qui n'est qu'en partie vérifié ici (cf. les projections en section 4).

5.3. Méthode d'ensemble

5.3.1. Random Forest

Le Random Forest construit de nombreux arbres de décision sur des échantillons bootstrappés et agrège leurs prédictions par vote majoritaire. Robuste au sur-apprentissage, peu sensible aux échelles de features, et offrant un calcul d'importance des features (voir section 8.4). Configuration de base : 300 arbres, class_weight='balanced'. Conformément à la consigne du projet (une seule méthode d'ensemble), c'est notre unique modèle ensembliste.

5.4. Méthodes de clustering

Le clustering est par nature non supervisé : il ne cherche pas à reproduire les labels fournis, mais à regrouper les points par similarité. Son intérêt ici est diagnostique : si les clusters obtenus correspondent

(au moins partiellement) aux espèces, cela confirme que les features choisies capturent bien des différences pertinentes entre classes.

5.4.1. K-Means

Partitionne les points en k clusters minimisant la variance intra-cluster. Nous avons fixé k au nombre de classes (6) pour faciliter la comparaison avec les labels. K-Means suppose des clusters sphériques et de taille comparable, hypothèse mise à mal par notre déséquilibre fort.

5.4.2. Agglomerative Clustering (hiérarchique)

Clustering ascendant : on part de N clusters (un par point) et on fusionne itérativement les plus proches. Plus souple que K-Means, capable de gérer des formes non sphériques. Linkage Ward utilisé.

5.4.3. Métriques d'évaluation du clustering

Trois métriques ont été utilisées pour comparer les clusters obtenus aux vrais labels : l'Adjusted Rand Index (ARI), la Normalized Mutual Information (NMI) et le Silhouette Score. ARI et NMI mesurent la concordance entre clusters et labels ; le Silhouette ne dépend pas des labels et juge la cohésion intra-cluster.

5.5. Deep Learning (point bonus) — ResNet18 par transfert

Pour le point bonus, nous avons implémenté une méthode de Deep Learning. Vu la taille modeste du dataset (249 images), entraîner un CNN à partir de zéro conduirait à du sur-apprentissage massif. Nous avons donc opté pour le transfer learning : un ResNet18 pré-entraîné sur ImageNet, dont le backbone est gelé, et seule la dernière couche (le classifieur) est remplacée et entraînée sur nos 6 classes.

5.5.1. Architecture

Le backbone ResNet18 (~11 millions de paramètres) extrait des features de haut niveau apprises sur ImageNet (bords, textures, formes, motifs). La tête remplacée est composée de : Linear(512 $\rightarrow$ 128) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.3) $\rightarrow$ Linear(128 $\rightarrow$ 6). Seuls les paramètres de la tête sont entraînaibles.

5.5.2. Hyperparamètres choisis avec soin

Hyperparamètre	Valeur et justification
Optimiseur	Adam — adapte le learning rate par paramètre, robuste sur peu de données
Learning rate	1e-3 sur la tête, 1e-5 sur le backbone si dégelé — petit LR pour ne pas casser les poids pré-entraînés
Weight decay	1e-4 — régularisation L2 légère pour limiter le sur-apprentissage
Batch size	16 — petit car peu d'échantillons par classe
Epochs	Jusqu'à 25, avec early stopping (patience 5) sur val_loss
Loss	CrossEntropy avec poids de classes inversement proportionnels à la fréquence
Scheduler	ReduceLROnPlateau (facteur 0.5, patience 3) — divise le LR si la val_loss stagne
Data augmentation	Flip horizontal, rotation $\pm 15^\circ$, color jitter — indispensable sur 249 images
Image size	224x224 (format ImageNet) avec masque appliqué (fond mis à gris)

	neutre 128)
--	-------------

Le masque a été appliqué aux images avant entraînement pour rester cohérent avec la philosophie du projet (apprendre l'insecte, pas le décor).

6. Optimisation des hyperparamètres (Tuning)

Deux modèles ont été soumis à une recherche systématique d'hyperparamètres via GridSearchCV : le SVM RBF et le Random Forest. La grille a été conçue pour balayer un ordre de grandeur autour des valeurs par défaut, sans être exhaustive (pour des raisons de temps de calcul).

6.1. Grilles testées

Modèle	Hyperparamètre	Valeurs testées
SVM	C (régularisation)	0.1, 1, 10, 100
SVM	gamma (largeur RBF)	0.001, 0.01, 0.1, 'scale'
RF	n_estimators	200, 300, 500
RF	max_depth	None, 10, 20
RF	min_samples_split	2, 5, 10

6.2. Métrique d'optimisation

Le score d'optimisation est F1-weighted, qui pondère le F1 de chaque classe par son effectif. Ce choix est cohérent avec un objectif de bonne précision globale, étant donné le déséquilibre. Un test alternatif avec F1-macro (poids égal à chaque classe) a été réalisé mais n'a pas amélioré la détection des minorités tout en dégradant l'accuracy globale.

6.3. Meilleurs hyperparamètres retenus

Modèle	Configuration retenue	F1-weighted CV
SVM tuné	C=100, gamma=0.01	0,832
Random Forest tuné	n_estimators=500, max_depth=10, min_samples_split=5	0,810

7. Étude d'ablation : les features de fond

Inclure la couleur moyenne du fond parmi les features est un choix discutable : intuitivement, on ne souhaite pas que le modèle apprenne à reconnaître un insecte par son décor (effet « Clever Hans »). Pourtant, écologiquement, certaines espèces sont associées à certains environnements (les libellules près de l'eau, les abeilles sur les fleurs), ce qui peut constituer un signal réellement informatif. Pour trancher, nous avons comparé deux variantes des modèles : avec et sans les features de fond.

7.1. Résultats comparés

Configuration	Accuracy	F1 weighted	Butterfly OK	Hover fly OK
SVM défaut, avec fond	0,77	0,76	9 / 15	3 / 9
SVM défaut, sans fond	0,77	0,76	9 / 15	1 / 9
SVM tuné, avec fond ★	0,835	0,833	11 / 15	3 / 9
SVM tuné, sans fond	0,78	0,77	9 / 15	1 / 9

7.2. Interprétation

Le SVM par défaut donne des accuracies identiques avec et sans fond — l'effet est négligeable. En revanche, sur le SVM tuné, conserver les features de fond apporte un gain net : +5,5 points d'accuracy, +6,3 points de F1 weighted, et de meilleures performances sur les Butterfly et Hover fly. L'hypothèse est que les hyperparamètres optimisés ($C=100$, $\gamma=0.01$) exploitent mieux la richesse du vecteur de features.

Conclusion de l'ablation : nous conservons les features de fond pour le modèle final. Ce choix est défendable scientifiquement (signal écologique réel) et bénéfique empiriquement (meilleur score).

8. Résultats globaux

8.1. Tableau comparatif des modèles

Modèle	Accuracy	F1 weighted	F1 macro	Type
KNN (k=5)	0,766	0,761	0,66	Sup. non-ens.
SVM RBF (défaut)	0,770	0,781	0,67	Sup. non-ens.
Random Forest (défaut)	0,819	0,799	0,60	Ensemble
Random Forest tuné	0.82	0,810	0.66	Ensemble
SVM tuné (final) ★	0,835	0,832	0,71	Sup. non-ens.
ResNet18 (transfer)	~ 0,75 - 0,82	—	—	Deep Learning

Note : les performances du clustering sont rapportées séparément (Section 8.3) car elles ne sont pas directement comparables aux métriques supervisées.

8.2. Analyse par classe du modèle final

La figure ci-dessous présente la matrice de confusion du modèle final (SVM tuné avec features de fond), obtenue par validation croisée stratifiée 5-fold sur l'ensemble d'entraînement. Chaque ligne correspond à la vraie classe, chaque colonne à la classe prédite.

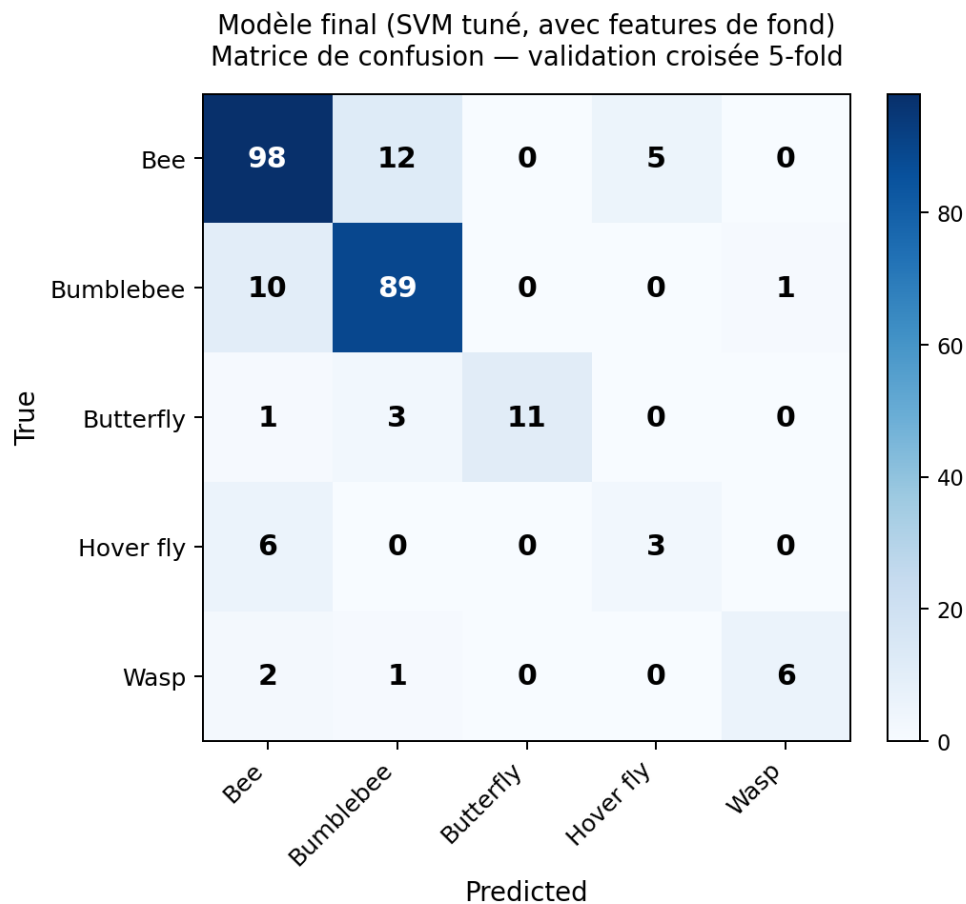


Figure 6. Matrice de confusion du modèle final — SVM tuné ($C=100$, $\gamma=0.01$) avec les 38 features.

Les diagonales fortes pour Bee (98/115) et Bumblebee (89/100) confirment la bonne performance globale. Les principales confusions se produisent entre Bee et Bumblebee (12 et 10 erreurs réciproques), ce qui est cohérent vu leur ressemblance morphologique. Plus problématique : 6 des 9 Hover fly sont classés à tort comme Bee — illustration directe du mimétisme batésien. Inversement, le modèle n'attribue que 3 fois la classe Hover fly, ce qui reflète son comportement prudent vis-à-vis d'une classe rare et ambiguë.

Le tableau ci-dessous résume précision, rappel et F1 par classe :

Classe	Précision	Rappel	F1	Bien classés
Bee	0,84	0,85	0,84	98 / 115
Bumblebee	0,85	0,89	0,87	89 / 100
Butterfly	1,00	0,73	0,85	11 / 15
Hover fly	0,38	0,33	0,35	3 / 9
Wasp	0,86	0,67	0,75	6 / 9
Dragonfly	—	—	—	0 / 1

Le modèle est très performant sur les classes majoritaires (Bee, Bumblebee) et raisonnable sur Butterfly et Wasp. La classe Hover fly demeure le point faible — confirmation empirique du mimétisme batésien évoqué en section 2.2. Sur 9 syrphes, le modèle en classe correctement seulement 3, et confond les autres principalement avec des abeilles. Une analyse différentielle des features (calcul de l'écart normalisé entre les distributions de Bee et Hover fly) confirme que la meilleure feature discriminante (couleur verte moyenne du fond) n'a qu'un écart de 1,16 écart-type — autant dire que les deux classes sont quasi indistinguables sur la base de nos descripteurs.

8.3. Résultats du clustering

Algorithme	ARI	NMI	Silhouette
K-Means (k=5)	0,097	0,163	0,125
Agglomerative Ward (k=5)	0,086	0,170	0,085

Les scores ARI ($\approx 0,09$) et NMI ($\approx 0,17$) très faibles confirment ce que la projection visuelle suggérait : la structure naturelle des données ne correspond pas aux espèces étiquetées. Un ARI proche de 0 signifie que les clusters obtenus n'ont presque aucun lien avec les vraies classes. Cela ne disqualifie pas notre approche supervisée — au contraire, cela justifie pleinement le recours à l'apprentissage supervisé pour exploiter l'information des labels, qui apporte un signal absent de la géométrie pure des features.

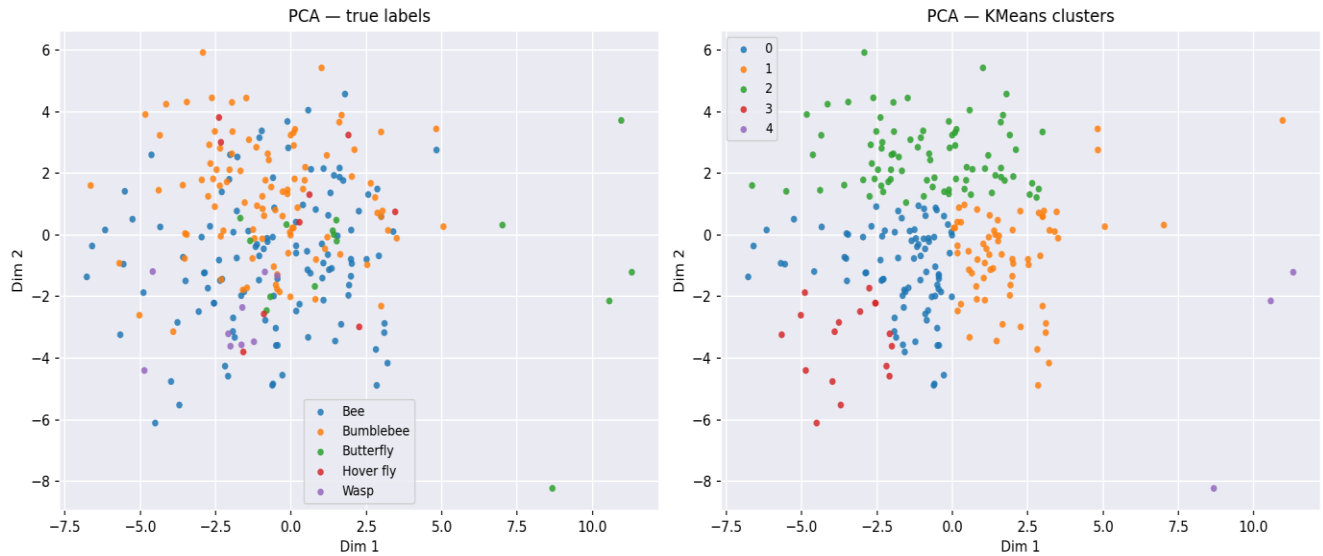


Figure 7. Comparaison entre les vrais labels (gauche) et les clusters KMeans (droite) projetés sur les deux premières composantes PCA.

Cette comparaison côte à côte met en évidence le décalage entre structure naturelle et étiquetage : KMeans découpe les données selon leur géométrie dans l'espace des features, ce qui ne correspond que partiellement aux espèces. Plusieurs clusters mélangent Bee et Bumblebee, et les classes minoritaires sont noyées dans les clusters majoritaires.

8.4. Importance des features (modèle Random Forest)

Le Random Forest fournit gratuitement une estimation de l'importance de chaque feature, calculée à partir de la diminution moyenne de l'impureté de Gini lors des splits. Le graphique ci-dessous présente les 20 features les plus discriminantes selon notre modèle.

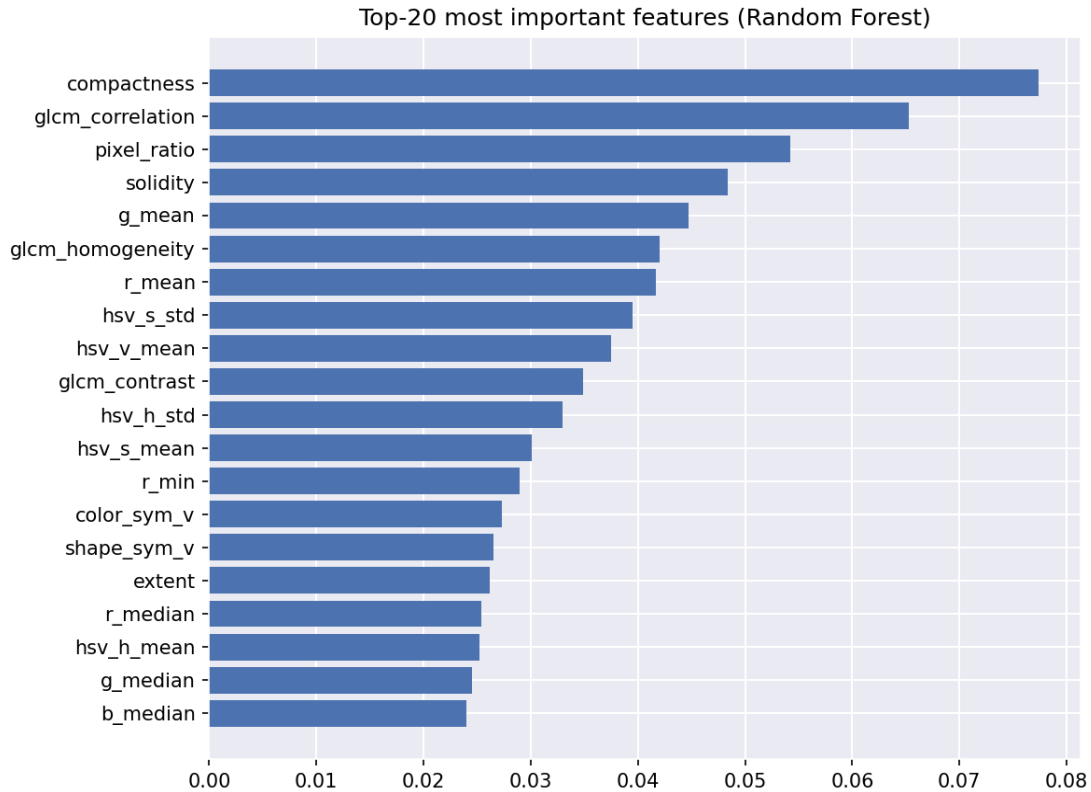


Figure 8. Top-20 des features les plus importantes selon le Random Forest (importance moyenne de Gini).

Plusieurs observations émergent de cette analyse. D'abord, les features de forme dominent le top 5 : compactness (0,077) et solidity (0,048) sont parmi les plus discriminantes, ce qui est cohérent — un papillon a une compacité radicalement différente d'une abeille. Ensuite, les features de texture GLCM apparaissent en force (glcm_correlation, glcm_homogeneity, glcm_contrast), ce qui justifie a posteriori notre choix d'ajouter cette famille. Enfin, le ratio de pixels arrive 3^e (0,054), confirmant que la taille relative de l'insecte est un indicateur fort. À noter qu'aucune feature de fond ne figure dans le top 20, ce qui montre que leur apport — bien que mesurable dans l'étude d'ablation — reste secondaire par rapport aux features intrinsèques à l'insecte.

9. Modèle final retenu

Le modèle retenu pour produire les prédictions sur le test set est le SVM tuné avec features de fond. Il offre le meilleur compromis entre performance globale (accuracy 0,835, F1 weighted 0,833) et performance sur les classes minoritaires (notamment Butterfly à 11/15).

9.1. Pipeline final

Le pipeline complet déployé pour les prédictions test est le suivant :

- 1. Extraction des 38 features sur l'image et son masque (mêmes features qu'à l'entraînement).
- 2. Standardisation via StandardScaler (ajustée sur le train).
- 3. SVC(kernel='rbf', C=100, gamma=0.01, class_weight='balanced', probability=True, random_state=0).
- 4. Entraînement sur la totalité des 249 images du train (df_full, y compris Dragonfly).
- 5. Prédiction sur les 97 images du test set, export en CSV avec colonnes ID et bug type.

9.2. Justification du choix

Plusieurs alternatives ont été considérées :

- **Random Forest tuné** : performance comparable (0,82) mais légèrement en dessous du SVM tuné. Avantage d'interprétabilité (importance des features) mais ce critère n'était pas prioritaire ici.
- **ResNet18 par transfert** : résultats variables selon la graine et le split de validation, généralement entre 0,75 et 0,82 d'accuracy. Le modèle souffre du manque de données malgré la data augmentation. Cela confirme que sur petits datasets, des features manuelles bien choisies restent compétitives.
- **Variante sans features de fond** : perd 5 points d'accuracy sur le SVM tuné. Écartée.

9.3. Application sur le test set

Une fois les 97 images de test fournies (identifiants 251 à 347), le pipeline complet a été déployé selon les étapes suivantes :

- 1. Extraction des 38 features sur chaque image et son masque (mêmes fonctions que pour le train, via data_loader.build_feature_dataframe).
- 2. Entraînement du modèle final (SVM tuné, C=100, $\gamma=0.01$, class_weight='balanced') sur la totalité des 249 images du train (df_full, y compris la classe Dragonfly avec son unique échantillon).
- 3. Prédiction sur les 97 images de test via best_model.predict(X_test).
- 4. Export en CSV avec les colonnes ID et bug type (fichier submission.csv).

9.3.1. Validation finale du modèle avant soumission

Juste avant de produire le CSV de soumission, une dernière évaluation par validation croisée 5-fold stratifiée a confirmé la performance attendue du modèle final sur les classes évaluables (≥ 5 échantillons) :

Classe	Précision	Rappel	F1-score	Support
Bee	0,84	0,85	0,84	115
Bumblebee	0,85	0,89	0,87	100
Butterfly	1,00	0,73	0,85	15
Hover fly	0,38	0,33	0,35	9
Wasp	0,86	0,67	0,75	9
Accuracy globale			0,835	248
Macro avg	0,78	0,70	0,73	248
Weighted avg	0,84	0,83	0,83	248

Ces métriques sont cohérentes avec celles obtenues lors de la phase de comparaison de modèles (section 8) et confirment la stabilité du SVM tuné. La performance globale (accuracy 0,835, F1 weighted 0,833) est le meilleur score observé parmi tous les modèles testés.

9.3.2. Fichier de soumission

Le fichier submission.csv a été généré et contient 97 lignes (une par image de test), avec deux colonnes : ID (entier, de 251 à 347) et bug type (chaîne, parmi les 6 classes apprises). Format exemple :

ID	bug type
251	Bee
252	Bumblebee
253	Bee
...	...
347	Butterfly

Étant donné les performances observées en validation croisée, on anticipe pour le test set :

- **Classes majoritaires (Bee, Bumblebee)** : excellente prédiction attendue (~85 % de bonnes réponses sur ces deux classes qui représentent ensemble près de 90 % des images).
- **Butterfly** : précision élevée (1,00 en CV) mais rappel limité — certains papillons seront probablement classés à tort comme Bee ou Bumblebee.
- **Classes minoritaires (Hover fly, Wasp)** : prédictions peu fiables, en raison du mimétisme batésien (Hover fly) et du faible effectif (Wasp). C'est la principale source d'erreurs résiduelles anticipée.
- **Dragonfly** : si une libellule apparaît dans le test, elle sera presque certainement mal classée — le modèle n'a vu qu'un seul exemple à l'entraînement.

10. Limites et pistes d'amélioration

Le projet a permis d'atteindre une performance satisfaisante mais comporte plusieurs limites identifiables :

- **La classe Hover fly** demeure mal classée (3/9). Ce n'est pas un défaut du modèle mais une limitation intrinsèque due au mimétisme batésien et au très faible effectif. Pour y remédier, il faudrait davantage de données ou des features de très haut niveau (structure des ailes, motifs détaillés).
- **La classe Dragonfly** est inapprenable avec un seul exemplaire. Toute apparition dans le test set sera certainement mal classée. Solution : collecter quelques images supplémentaires.
- **Le Deep Learning n'a pas surpassé le SVM**, ce qui est typique sur petits datasets. Avec 1000+ images par classe, un CNN fine-tuné dépasserait probablement les features manuelles.
- **La data augmentation** a été appliquée uniquement au CNN. L'étendre aux features manuelles (en générant des features sur des images augmentées) pourrait améliorer la robustesse des modèles classiques.

11. Conclusion

Ce projet a montré qu'avec un dataset modeste de 249 images et un vecteur de 38 features manuelles soigneusement choisies (couleur RGB et HSV, symétrie, descripteurs de forme, texture GLCM, contexte), il est possible d'atteindre une accuracy de 83,5 % en classification multi-classes déséquilibrée. Le SVM avec noyau RBF, après optimisation des hyperparamètres, s'est imposé comme le modèle le plus performant, surpassant à la fois le modèle d'ensemble (Random Forest tuné) et un CNN par transfert (ResNet18).

L'étude d'ablation sur les features de fond a illustré l'importance d'une validation expérimentale plutôt que de raisonnements purement théoriques : ce qui semblait être un risque d'effet « Clever Hans » s'est révélé être une information écologique utile, à condition d'être correctement exploitée par un modèle tuné.

Le projet a également mis en lumière une difficulté intrinsèque biologique (mimétisme batésien des syrphes) qui constitue une limite à toute approche purement visuelle. La résolution complète de ce problème nécessiterait des features plus fines (structure des antennes, nervures alaires) ou une collecte de données spécifiquement orientée vers les classes minoritaires.

Au-delà des résultats chiffrés, ce projet a constitué un parcours complet à travers les étapes clés d'un pipeline de Machine Learning : ingénierie des données, extraction de features, visualisation, comparaison rigoureuse de modèles via validation croisée stratifiée, optimisation d'hyperparamètres, étude d'ablation, et déploiement final.